Question 1

```
import java.util.Arrays;


public class Student implements Comparable {
   private String lastName;
   private int idNumber;


   public Student(String lastName, int idNumber) {
      this.lastName = lastName;
      this.idNumber = idNumber;
   }


   // Accessors
   public String getLastName() { return lastName; }
   public int getIdNumber() { return idNumber; }


   // Mutators
   public void setLastName(String lastName) { this.lastName = lastName; }
   public void setIdNumber(int idNumber) { this.idNumber = idNumber; }


   @Override
   public int compareTo(Object o) {
      Student other = (Student) o;
      // Sort by last name (lexicographic)
      return this.lastName.compareTo(other.lastName);
```

```java
        // To sort by ID instead, use:
        // return this.idNumber - other.idNumber;
    }

    public static void main(String[] args) {
        Student[] students = new Student[5];
        students[0] = new Student("Bobby", 110);
        students[1] = new Student("Tom", 1);
        students[2] = new Student("Alice", 300);
        students[3] = new Student("Dan", 20);
        students[4] = new Student("Elf", 5);

        Arrays.sort(students);

        for (Student s : students) {
            System.out.println(s.getLastName() + " " + s.getIdNumber());
        }
    }
}
```

Question 2

Class: Movie

Movie is a **concrete** class. It has no superclass beyond the default `Object` class that all Java classes inherit from. Its subclass is `ActionMovie`.

Variables:

- `MovieID` — private integer that stores the unique identifier for a movie.
- `title` — private String that stores the title of the movie.
- `rating` — private String that stores the rating of the movie (e.g. PG, R, etc.).

Methods:

- `Movie()` — the default constructor. Takes no input. Initializes `MovieID` to 0, `title` to an empty string, and `rating` to an empty string. Returns nothing.
- `Movie(int ID, String title, String rating)` — the parameterized constructor. Takes an integer ID, a String title, and a String rating as input. Sets the instance variables to the provided values. Returns nothing.
- `getID()` — takes no input. Returns the integer value of `MovieID`.
- `setID(int ID)` — takes an integer as input. Sets `MovieID` to the provided value. Returns nothing.
- `getTitle()` — takes no input. Returns the String value of `title`.
- `setTitle(String name)` — takes a String as input. Sets `title` to the provided value. Returns nothing.
- `getRating()` — takes no input. Returns the String value of `rating`.
- `setRating(String rating)` — takes a String as input. Sets `rating` to the provided value. Returns nothing.
- `equals(Object other)` — takes any Object as input. Checks if the other object is null, then checks if it's the same class, then compares `MovieID` values. Returns a boolean — `true` if the two movies share the same ID, `false` otherwise.
- `calcLateFees(int daysLate)` — takes an integer representing the number of days late as input. Calculates the late fee by multiplying 2.0 by the number of days late. Returns a double.

Class: ActionMovie

ActionMovie is a **concrete** class. Its superclass is `Movie`. It has no subclasses. It inherits all variables and methods from `Movie` but overrides `calcLateFees`.

Variables:

ActionMovie has no variables of its own. It inherits `MovieID`, `title`, and `rating` from `Movie`, though they are private and only accessible through the inherited getters and setters.

Methods:

- `ActionMovie()` — the default constructor. Takes no input. Calls `super()` to invoke Movie's default constructor. Returns nothing.
- `ActionMovie(int ID, String title, String rating)` — the parameterized constructor. Takes an integer ID, a String title, and a String rating as input. Calls `super(ID, title, rating)` to pass those values up to Movie's constructor. Returns nothing.
- `calcLateFees(int daysLate)` — **overrides** Movie's version. Takes an integer representing days late as input. Calculates the late fee at a higher rate of 3.0 per day instead of 2.0. Returns a double. This reflects that action movies cost more to rent and therefore have higher late fees.

Class: ComedyMovie

ComedyMovie is a **concrete** class. Its superclass is `Movie`. It has no subclasses. Like ActionMovie, it inherits all variables and methods from `Movie` but overrides `calcLateFees`.

Variables:

ComedyMovie has no variables of its own. It inherits `MovieID`, `title`, and `rating` from `Movie`, though they are private and only accessible through the inherited getters and setters.

Methods:

- `ComedyMovie()` — the default constructor. Takes no input. Calls `super()` to invoke Movie's default constructor. Returns nothing.
- `ComedyMovie(int ID, String title, String rating)` — the parameterized constructor. Takes an integer ID, a String title, and a String rating as input. Calls `super(ID, title, rating)` to pass those values up to Movie's constructor. Returns nothing.
- `calcLateFees(int daysLate)` — **overrides** Movie's version. Takes an integer representing days late as input. Calculates the late fee at a rate of 2.5 per day, which is between Movie's base rate of 2.0 and ActionMovie's rate of 3.0. Returns a double.

Class: DramaMovie

DramaMovie is a **concrete** class. Its superclass is `Movie`. It has no subclasses. Unlike ActionMovie and ComedyMovie, it does **not** override `calcLateFees`, meaning it uses Movie's default late fee calculation.

Variables:

DramaMovie has no variables of its own. It inherits `MovieID`, `title`, and `rating` from `Movie`, though they are private and only accessible through the inherited getters and setters.

Methods:

- `DramaMovie()` — the default constructor. Takes no input. Calls `super()` to invoke Movie's default constructor. Returns nothing.
- `DramaMovie(int ID, String title, String rating)` — the parameterized constructor. Takes an integer ID, a String title, and a String rating as input. Calls `super(ID, title, rating)` to pass those values up to Movie's constructor. Returns nothing.
- `calcLateFees(int daysLate)` — DramaMovie does **not** override this method, so it inherits Movie's version directly. It takes an integer representing days late and returns a double calculated as 2.0 times the number of days late.

Question 3

If Kill Bill: Volume 2 is 2 days late the fee is 6.0

If Kill Bill: Volume 2 is 3 days late the fee is 9.0

If Mean Girls is 3 days late the fee is 7.5

If Mystic River is 3 days late the fee is 6.0

Is Kill Bill: Volume 2 equal Mystic River? false

Is Mean Girls equal Mystic River? false

Is Mystic River equal Mystic River, Second Copy? true

Question 4

the Rental class is missing, its code is as below:

```
class Rental {
    private Movie movie;
    private int daysRented;
    private int daysLate;

    public Rental(Movie movie, int daysRented) {
        this.movie = movie;
        this.daysRented = daysRented;
        this.daysLate = 0;
    }

    public Movie getMovie() {
        return movie;
    }

    public void setMovie(Movie movie) {
        this.movie = movie;
    }

    public int getDaysRented() {
        return daysRented;
    }

    public void setDaysRented(int daysRented) {
```

```
        this.daysRented = daysRented;
    }

    public int getDaysLate() {
        return daysLate;
    }

    public void setDaysLate(int daysLate) {
        this.daysLate = daysLate;
    }

    public double getLateFees() {
        return movie.calcLateFees(daysLate);
    }
}
```

Question 5

```
import java.util.ArrayList;

class RandomDrawing<T>
{
   private ArrayList<T> box;

   public RandomDrawing()
   {
      box = new ArrayList<T>();
   }

   public void add(T obj)
   {
      box.add(obj);
   }

   public boolean isEmpty()
   {
      return (box.size() == 0);
   }

   public T drawItem()
   {
      if (isEmpty()) return null;
      int randIndex = (int) (Math.random() * box.size());
```

```
        T item = box.get(randIndex);
        box.remove(item);
        return item;
    }
}

public class Question5RandomDrawing
{
    public static void main(String[] args)
    {
        RandomDrawing<Integer> intBox = new RandomDrawing<Integer>();
        RandomDrawing<String> stringBox = new RandomDrawing<String>();

        intBox.add(3);
        intBox.add(10);
        intBox.add(75);
        intBox.add(45);

        stringBox.add("Carol");
        stringBox.add("Bob");
        stringBox.add("Ted");
        stringBox.add("Alice");

        System.out.println("Drawing from the box of integers:");
        while (!intBox.isEmpty())
        {
            System.out.println(intBox.drawItem());
```

```java
        }

        System.out.println("Drawing from the box of strings:");
        while (!stringBox.isEmpty())
        {
           System.out.println(stringBox.drawItem());
        }
    }
}
```